

Homelab-Projekt: Von Proxmox zur automatisierten Infrastruktur

Ansible-Einstieg, Netzwerksegmentierung & Dynamic Inventory — Zusammenfassung der heutigen Session

Diese Dokumentation fasst zusammen, was in dieser Session aufgebaut wurde: ein Ansible-Control-Node in einem Proxmox-Homelab, die Automatisierung von Services über mehrere Netzwerksegmente hinweg, und eine dynamische Inventarisierung direkt aus der Proxmox-API. Ziel war es, den Übergang von manueller Systemadministration hin zu Infrastructure-as-Code (IaC) praktisch zu erlernen — als Vorbereitung auf eine Rolle im Bereich System Engineering / Infrastruktur.

Ausgangslage

- Proxmox VE Homelab mit mehreren LXC-Containern (Nextcloud, Immich, AdGuard, Uptime-Kuma, Dashboard u.a.) und einer OPNsense-Firewall-VM
- Zwei Netzwerksegmente: **vmbr0** (Fritzbox-Netz, 192.168.178.x) und **vmbr1** (internes LAN hinter OPNsense, 192.168.1.x)
- Ziel: Services nicht mehr manuell per GUI verwalten, sondern reproduzierbar per Ansible automatisieren

Ergebnis auf einen Blick

Baustein	Status
Ansible-Control-Node (LXC 108) aufgesetzt	erledigt
SSH-Key-Authentifizierung eingerichtet	erledigt
Netzwerksegmentierung verstanden & Firewall-Regel gesetzt	erledigt
Erstes Playbook geschrieben & ausgeführt	erledigt
Playbook mit Variablen, Handlern, Bedingungen erweitert	erledigt
Multi-Host-Ausführung (mehrere Server gleichzeitig)	erledigt
Dynamic Inventory aus Proxmox-API	erledigt
SSH-Key automatisiert auf alle Hosts verteilt	6 von 8 Hosts erledigt

1. Ansible-Control-Node aufsetzen

Zuerst wurde ein eigener, unprivilegierter LXC-Container (ID 108, Debian 12, Hostname `Ansible-control`) angelegt. Dieser Container dient als zentraler Ausgangspunkt, von dem aus alle anderen Server verwaltet werden — er läuft selbst nicht als Produktivsystem, sondern nur als Werkzeug.

Warum ein eigener Container?

Automatisierungs-Tools sollten nicht auf einem produktiven System (z.B. Nextcloud) laufen, sondern getrennt — das verhindert, dass ein Fehler beim Testen gleich einen laufenden Dienst stört.

Ansible installieren

```
apt update
apt install -y ansible
```

Prüfen der Installation:

```
ansible --version
```

2. SSH-Key-Authentifizierung

Damit Ansible sich automatisiert (ohne Passwort-Eingabe) mit den Zielsevern verbinden kann, wurde ein SSH-Schlüsselpaar erzeugt und der öffentliche Schlüssel auf die Zielsysteme verteilt.

```
# Schlüsselpaar erzeugen (auf Ansible-Control)
ssh-keygen -t ed25519 -C "ansible-control"

# Öffentlichen Schlüssel auf einen Zielserver kopieren
ssh-copy-id root@<ZIEL-IP>

# Testen, ob Login ohne Passwort funktioniert
ssh root@<ZIEL-IP>
```

Wichtiger Stolperstein: Bei manchen Debian-Containern ist Root-Login per Passwort standardmäßig deaktiviert (`PermitRootLogin prohibit-password`). Fix direkt auf dem Zielsystem:

```
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config
systemctl restart ssh
```

3. Netzwerksegmentierung & Firewall-Regel (OPNsense)

Das Homelab besteht aus zwei getrennten Netzen: dem Fritzbox-Netz (178.x, WAN-Seite von OPNsense) und dem internen LAN hinter OPNsense (1.x). Verwaltungs-Tools wie Ansible-Control und Uptime-Kuma wurden bewusst ins **interne LAN** gelegt — das entspricht dem Prinzip, wie es auch in Unternehmen gemacht wird: Management-Systeme gehören ins geschützte Netz, nicht ins Netz Richtung Internet.

AdGuard blieb bewusst im 178.x-Netz, da es als DNS-Filter für *alle* Geräte im Haushalt erreichbar sein muss. Damit Ansible-Control (LAN) dennoch gezielt auf AdGuard (WAN-Seite) zugreifen kann, wurde in OPNsense eine **präzise, minimale Firewall-Regel** angelegt — statt pauschal ganze Netze zu öffnen (Prinzip: *Least Privilege*).

Firewall-Regel in OPNsense (Firewall → Rules → LAN)

- Action: Pass

- Protocol: TCP, Ziel-Port: 22 (SSH)
- Source: einzelne IP von Ansible-Control
- Destination: einzelne IP von AdGuard

Nach dem Speichern: „Änderungen übernehmen / Apply“ nicht vergessen.

4. Das erste Playbook: Inventory & Ping-Test

Ein Ansible-„Inventory“ ist die Liste der Server, die verwaltet werden sollen. Zum Einstieg wurde eine einfache, manuell gepflegte Datei genutzt:

```
mkdir -p ~/ansible-project && cd ~/ansible-project
nano inventory.ini
[adguard]
192.168.178.105 ansible_user=root

[monitoring]
192.168.1.3 ansible_user=root

[all_my_vms:children]
adguard
monitoring
```

Erreichbarkeit testen (kein Playbook, nur ein Verbindungstest):

```
ansible -i inventory.ini all_my_vms -m ping
Erwartetes Ergebnis pro Host: "ping": "pong"
```

5. Playbook mit Logik: Variablen, Handler, Bedingungen

Aufbauend auf dem Ping-Test wurde ein echtes Playbook geschrieben, das Pakete aktualisiert und den Status eines Dienstes prüft. Dabei wurden zentrale Ansible-Konzepte verwendet:

- **vars** — wiederverwendbare Werte (z.B. Name des zu prüfenden Dienstes)
- **register** — Ergebnis eines Befehls in einer Variable speichern
- **debug** — Werte/Status zur Kontrolle ausgeben
- **handlers + notify** — Aktionen, die nur bei tatsächlicher Änderung ausgeführt werden

```
---
- name: AdGuard warten und Status prüfen
  hosts: adguard
  become: true
  vars:
    service_name: AdGuardHome

  tasks:
    - name: Paketliste aktualisieren
      apt:
        update_cache: yes

    - name: Pakete upgraden
      apt:
        upgrade: dist
      notify: Neustart Hinweis

    - name: Prüfen ob Dienst läuft
      command: systemctl is-active {{ service_name }}
      register: service_status
      changed_when: false
      failed_when: false

    - name: Status ausgeben
      debug:
```

```
msg: "Service Status: {{ service_status.stdout }}"

handlers:
- name: Neustart Hinweis
  debug:
    msg: "Es gab Updates - eventuell Neustart nötig!"
```

Wichtiger Stolperstein: Bei minimalen LXC-Containern ist sudo teils nicht installiert. Da der SSH-Login ohnehin direkt als root erfolgte, wurde `become: true` entfernt — root-Rechte waren bereits vorhanden.

6. Mehrere Server gleichzeitig automatisieren

Ansible führt Tasks standardmäßig parallel auf allen passenden Hosts aus. Um das Playbook auf mehrere Server gleichzeitig laufen zu lassen, genügt es, im Playbook eine Gruppe statt eines einzelnen Hosts anzugeben:

```
hosts: all_my_vms # statt: hosts: adguard
```

Ergebnis: Das Update-Playbook lief in einem Durchgang auf AdGuard (178.x-Netz) und Uptime-Kuma (1.x-Netz) — trotz unterschiedlicher Netzwerksegmente, dank der zuvor eingerichteten Firewall-Regel und SSH-Zugriffe.

7. Dynamic Inventory: Automatische Serverliste aus Proxmox

Der wichtigste Schritt für Skalierbarkeit: Statt Server-IPs manuell in einer Datei zu pflegen, liest Ansible sie live über die Proxmox-API aus. Neue VMs/Container erscheinen automatisch — das ist der Ansatz, der auch bei sehr vielen Servern (z.B. 200 VMs in einem Unternehmen) funktioniert.

Vorbereitung

```
# Community-Collection installieren (aktuelle, nicht-deprecated Version)
ansible-galaxy collection install community.proxmox

# Python-Abhängigkeiten für den API-Zugriff
apt install -y python3-pip
pip3 install proxmoxer requests --break-system-packages
```

API-Token in Proxmox erstellen

Proxmox-Weboberfläche → Rechenzentrum → Berechtigungen → API-Tokens → „+ Hinzufügen“:

- Benutzer: root@pam
- Token-ID: ansible
- Privilegentrennung: deaktiviert (Häkchen entfernt)
- Den angezeigten Secret-Wert sofort sichern — wird danach nicht mehr angezeigt

Inventory-Konfigurationsdatei

```
cd ~/ansible-project
nano proxmox.yml
plugin: community.proxmox.proxmox
url: https://<PROXMOX-HOST-IP>:8006
user: root@pam
token_id: ansible
token_secret: <TOKEN-SECRET>
validate_certs: false
want_facts: true
compose:
  ansible_host: proxmox_net0.ip.split('/')[0]
```

Die letzte Zeile (compose) ist entscheidend: Proxmox liefert von sich aus nur Hostnamen, keine feste Verbindungs-IP. Ohne diesen Zusatz versucht Ansible, den Hostnamen per DNS aufzulösen und scheitert. Der Ausdruck extrahiert die IP-Adresse aus den Proxmox-Netzwerkdaten (z.B. aus 192.168.178.105/24 wird 192.168.178.105).

Testen

```
ansible-inventory -i proxmox.yml --list
ansible-inventory -i proxmox.yml --host adguard
```

Ergebnis: Ansible erkannte automatisch alle Systeme aus Proxmox — inklusive automatisch gebildeter Gruppen wie proxmox_all_lxc (alle Container), proxmox_all_qemu (alle VMs) und proxmox_all_running (alles, was aktuell läuft) — ganz ohne manuelle Pflege.

```
ansible-playbook -i proxmox.yml site.yml --limit proxmox_all_lxc
```

Bekannter Sonderfall: Hosts mit DHCP statt fester IP

Ein Container (KI-Agents) war in Proxmox mit `ip: dhcp` statt einer festen IP konfiguriert. Der `compose`-Ausdruck lieferte dadurch wörtlich `"dhcp"` statt einer Adresse. Ansible versuchte dies als Hostnamen aufzulösen und scheiterte. **Nächster Schritt (offen):** die echte IP stattdessen aus `proxmox_lxc_interfaces` auslesen.

8. SSH-Keys automatisiert auf alle Hosts verteilen

Statt für jeden neuen Server einzeln `ssh-copy-id` auszuführen, wurde ein eigenes Playbook gebaut, das den öffentlichen Schlüssel per einmaliger Passwort-Authentifizierung auf alle Proxmox-Container verteilt:

```
apt install -y sshpass # wird für passwortbasierte Verbindungen benötigt
---
- name: SSH-Key auf alle LXCs verteilen
  hosts: proxmox_all_lxc
  gather_facts: false
  vars:
    ansible_ssh_pass: "<ROOT-PASSWORT>"
  tasks:
    - name: Public Key kopieren
      authorized_key:
        user: root
        key: "{{ lookup('file', '/root/.ssh/id_ed25519.pub') }}"
ansible-playbook -i proxmox.yml copy-key.yml --limit proxmox_all_lxc
```

Zusätzlich wurde die interaktive SSH-Host-Key-Abfrage deaktiviert, damit automatisierte Läufe nicht hängen bleiben (Datei `~/.ansible.cfg`):

```
[defaults]
```

Sicherheitshinweis: Diese Einstellung ist für ein vertrauenswürdiges Homelab akzeptabel. In einer Unternehmensumgebung würde man Host-Keys stattdessen sauber vorab verteilen (z.B. über eine SSH Certificate Authority) statt die Prüfung pauschal abzuschalten.

Ergebnis: 6 von 8 Containern erfolgreich eingerichtet. Zwei offene Punkte für die nächste Session:

- **business-sites:** abweichendes root-Passwort — Passwort prüfen/zurücksetzen oder Key manuell kopieren
- **KI-Agents:** DHCP-Sonderfall (siehe oben) — `compose`-Ausdruck muss angepasst werden

9. Zentrale Konzepte kurz erklärt

Begriff	Bedeutung
Inventory	Liste der Server, die Ansible verwalten soll — statisch (Datei) oder dynamisch (z.B. aus Proxmox-API)
Playbook	YAML-Datei mit den Arbeitsschritten ("Tasks"), die auf den Ziel-Hosts ausgeführt werden
Task	Ein einzelner Arbeitsschritt innerhalb eines Playbooks (z.B. "Pakete aktualisieren")
Module	Fertiger Baustein für eine bestimmte Aktion, z.B. apt (Pakete) oder authorized_key (SSH-Keys)
Handler	Task, der nur ausgeführt wird, wenn ein anderer Task per notify eine Änderung gemeldet hat
Vars	Wiederverwendbare Variablen innerhalb eines Playbooks
become	"Werde root" (via sudo) — nicht nötig, wenn ohnehin direkt als root verbunden wird
Idempotenz	Ein Playbook mehrfach auszuführen verändert nichts mehr, sobald der Zielzustand erreicht ist

10. Warum das für Unternehmen relevant ist

Das heute gebaute Setup ist im Kern bereits das Fundament, das auch bei sehr großen Infrastrukturen (z.B. 200+ VMs) verwendet wird — nur mit zusätzlicher Struktur:

- **Gruppen & Rollen** statt einzelner Playbooks pro Server — wiederverwendbare Bausteine für z.B. "Webserver" oder "Datenbank"
- **Dynamic Inventory** statt statischer Listen — bereits heute umgesetzt
- **Ansible Vault** zur verschlüsselten Verwaltung von Passwörtern/Secrets, statt Klartext in Dateien
- **CI/CD-Pipelines** (z.B. GitHub Actions), die Playbook-Änderungen automatisch testen und ausrollen
- **SSH Certificate Authority** statt einzeln verteilter Schlüssel, für sauberes Zugriffsmanagement im großen Maßstab

Für Bewerbungsgespräche im Bereich System Engineering ist dieses Projekt eine konkrete, vorzeigbare Geschichte: Netzwerksegmentierung verstanden und umgesetzt, gezielte statt pauschale Firewall-Regeln, Infrastructure-as-Code mit Ansible, und ein Verständnis dafür, wie sich das Prinzip auf sehr viele Server skalieren lässt.

11. Offene nächste Schritte

- SSH-Key-Verteilung für business-sites und KI-Agents abschließen
- compose-Ausdruck in proxmox.yml robuster machen (DHCP-Fall abfangen)
- Playbooks in Rollen aufteilen (z.B. eine "common"-Rolle für alle Hosts)
- Projekt mit README auf GitHub dokumentieren
- Anschließend: Monitoring mit Grafana aufbauen

Zusammenfassung erstellt als Lern- und Interview-Dokumentation für das Homelab-Projekt.